

Hardware Business Management System

Purpose of This Document

This Software Design Document describes how the Hardware Business Management System should be designed from the approved requirements. It is intended for software developers, Cloud AI builders, database planners, UI designers, testers, and technical reviewers. The document translates the SRS into a practical design covering architecture, modules, data design, synchronization, security, error handling, reporting, deployment, and technical acceptance expectations.

How to Use This Document

- Use Confirmed items as the expected design direction for the first implementation.
- Use Proposed items as a recommended design that can still be changed during technical planning.
- Use Pending Decision items as questions for the business owner before hard coding business logic.
- Do not finalize profit, stock valuation, 60/40 allocation destinations, tax, or debt accounting rules until the owner confirms them.

Status Label Convention

Confirmed means the item is part of the agreed scope. Proposed means it is a recommended implementation detail that can still change. Pending Decision means the business owner must decide before a developer or Cloud AI treats it as final.

Status	Meaning	Developer Handling
Confirmed	Part of the agreed project scope.	Implement unless later changed by the owner.
Proposed	Recommended design or implementation approach.	Validate during technical planning.
Pending Decision	Business rule or technical rule not fully decided.	Ask the owner or make configurable. Do not hard code.

Design Overview

The system design is offline-first for shop operations and centralized for administration. The mobile app records core transactions locally using SQLite, then synchronizes with a central backend and MySQL database when connectivity is available. The web admin provides central management, reporting, configuration, and audit review. The design must support multiple shops, Super Admin and Shop Admin roles, stock movement, purchases, sales, expenses, capital, debts, profit reporting, 60/40 allocation, notifications, and auditability.

Design Area	Design Direction	Status
Application style	Mobile app plus web admin connected to a shared backend API.	Confirmed
Mobile persistence	SQLite local database for offline shop operations.	Confirmed
Central persistence	MySQL database for synchronized business records and reports.	Confirmed
Backend	API layer for authentication, authorization, validation, reports, sync, and audit logging.	Proposed
Synchronization	Queue-based local changes synced to central database when online.	Confirmed
Accounting logic	Profit and allocation logic should be configurable until final rules are approved.	Pending Decision
Security	Role-based permissions, password hashing, protected transport, and audit trail.	Confirmed

System Architecture

Logical Architecture

Layer	Responsibilities	Status
Presentation layer	Mobile screens for shop work and web screens for administration and reporting.	Confirmed
Application layer	Business workflows such as sales, purchases, expenses, debts, capital, daily opening, and daily closing.	Confirmed
API layer	Receives requests, validates permissions, applies business rules, returns responses, and writes audit logs.	Proposed
Synchronization layer	Handles pending mobile records, retries, duplicate prevention, conflict detection, and server acknowledgement.	Confirmed
Data layer	SQLite for local mobile storage and MySQL for central storage.	Confirmed
Notification layer	Creates and delivers stock, debt, sync, and daily closing reminders.	Confirmed
Reporting layer	Aggregates sales, purchases, stock, expense, debt, capital, profit, closing, and audit data.	Confirmed

Recommended Component Flow

- User opens the mobile app or web admin and authenticates.
- The client stores or requests data through the backend API, depending on platform and connectivity.
- The mobile app writes offline transactions to SQLite with pending sync status when internet is unavailable.
- When online, mobile sync sends pending records to the backend API.
- The backend validates records, checks duplicates, applies permissions, writes to MySQL, and returns sync acknowledgements.
- Reports and dashboards read from the central MySQL database.
- Audit logs record critical actions from mobile, web, backend, and synchronization processes.

Module Design

Module	Main Responsibilities	Key Design Notes	Status
Authentication and users	Login, session handling, role assignment, shop assignment, active or inactive users.	Use role and shop permissions on every protected screen and API route.	Confirmed

Module	Main Responsibilities	Key Design Notes	Status
Shop management	Create and manage shops, assign users, view shop performance.	Super Admin controls this mainly through web admin.	Confirmed
Product management	Products, categories, units, buying price, selling price, reorder level, status.	Product code or SKU should be unique if used.	Confirmed
Purchases	Record purchased items, supplier if used, costs, payment status, stock increase.	Purchase item rows should generate stock movement records.	Confirmed
Sales	Record sold items, quantities, selling price, payment status, customer if used, stock decrease.	Sales should generate stock movement and support offline entry.	Confirmed
Stock	Current stock, stock movements, low stock alerts, adjustments, stock reports.	Stock movement history should be append-only for traceability.	Confirmed
Expenses	Expense category, amount, shop, date, reason, user, report visibility.	Whether each category reduces profit is pending decision.	Pending Decision
Capital	Capital injections, withdrawals, balance movement, reports.	Exact relationship between capital and profit allocation must be confirmed.	Pending Decision
Debts	Customer or supplier debts, repayments, outstanding balance, reports.	Debt categories and due-date policy should be confirmed.	Pending Decision
Profit and 60/40 allocation	Calculate approved profit and split into 60 percent and 40 percent outputs.	Formula, timing, and destination labels must be confirmed by owner.	Pending Decision
Daily opening and closing	Open business day, attach transactions, review totals, close day with notes.	Closing corrections should be audited instead of silently replacing history.	Confirmed
Reports	Sales, purchase, stock, expenses, capital, debts, profit, closing, and audit reports.	Web admin should provide full reports and export where required.	Confirmed
Notifications	Low stock, debt reminders, sync failures, closing reminders, admin alerts.	Notification channels can be in-app first, then push or SMS later if needed.	Proposed
Audit trail	Critical actions, changes, approvals, deletions, logins, sync results.	Audit records should not be editable by normal users.	Confirmed

Data Design

The database design should preserve accurate transaction history, stock traceability, synchronization safety, and report correctness. The physical schema can be refined during implementation, but the following entity groups should be represented in both the central database and the local mobile database where offline use requires them.

Entity Group	Central MySQL Design	Mobile SQLite Design	Status
Users roles shops	Authoritative users, roles, permissions, shop assignments.	Cached authenticated user, permissions, assigned shop data.	Confirmed
Products categories	Authoritative product catalog, prices, categories, reorder levels.	Cached product catalog needed for offline transactions.	Confirmed
Sales and sale items	Final synchronized sales records and report source.	Local sales queue with sync status and server id after sync.	Confirmed
Purchases and purchase items	Final synchronized purchase records and stock source.	Local purchase queue where offline purchase entry is allowed.	Confirmed
Stock balances movements	Central stock balance and full movement history.	Local stock snapshot plus pending movement records.	Confirmed
Expenses	Central expense records and report source.	Local expense queue with sync status.	Confirmed
Capital	Central capital movement records.	Local queue if Shop Admin is allowed to record capital movement.	Confirmed
Debts repayments	Central debt ledger and repayment history.	Local debt and repayment queue if offline debt recording is allowed.	Confirmed
Daily sessions	Business day opening, closing, totals, and notes.	Local active day and closing queue.	Confirmed
Audit logs	Central audit log for critical actions.	Local action log for offline actions until synchronized.	Confirmed
Sync queue	Server sync receipts, conflict records, and rejected payloads.	Pending local operations, retries, errors, and server acknowledgements.	Confirmed

Database Design Rules

- Use stable unique identifiers for records created offline so retries do not create duplicates.
- Use created_at, updated_at, created_by, updated_by, shop_id, and sync_status fields where relevant.

- Use soft delete or active status for records that have transaction history.
- Use append-only movement tables for stock and important financial ledger activity.
- Store before and after values in audit logs when critical records are changed.
- Store the calculation configuration or formula version used for profit and allocation outputs after rules are confirmed.

API and Service Design

The backend API should provide clear, versioned endpoints for mobile and web clients. Exact endpoint paths can change during implementation, but the service responsibilities below should be covered.

Service Area	Example Responsibilities	Status
Auth service	Login, logout, token refresh, password reset, user session validation.	Confirmed
User and role service	Create users, assign roles, assign shops, deactivate users, enforce permissions.	Confirmed
Catalog service	Products, categories, units, prices, reorder levels, active status.	Confirmed
Transaction service	Sales, purchases, expenses, debts, repayments, capital entries, daily sessions.	Confirmed
Stock service	Stock balance, movement history, low stock checks, adjustments.	Confirmed
Sync service	Receive local payloads, detect duplicates, return server ids, report conflicts and errors.	Confirmed
Report service	Filtered reports and export-ready datasets.	Confirmed
Notification service	Create, list, read, and resolve notifications.	Confirmed
Audit service	Write and query audit records for authorized users.	Confirmed

API Response Principles

- Return clear success, validation error, permission error, duplicate, conflict, and server error responses.
- Never expose sensitive internal server traces to ordinary users.
- Return field-level validation messages where the user can correct the issue.
- For sync requests, return server record ids, accepted records, rejected records, retryable failures, and conflict details.
- Use pagination and filters for large reports, audit logs, products, and transaction history.

Offline and Synchronization Design

Offline behavior is a central design requirement. The mobile app must remain useful in the shop even when internet connectivity is unstable. The system must avoid data loss and must prevent duplicate counting when a transaction is retried.

Sync Step	Design Requirement	Status
Local save	Mobile app saves the transaction to SQLite first with pending sync status.	Confirmed
Local validation	Mobile app validates required fields, quantities, prices, dates, and permissions before saving where possible.	Confirmed
Sync trigger	Sync runs when connectivity returns, when user manually retries, or on scheduled background intervals if supported.	Proposed
Server validation	Backend validates user, shop, payload, business rules, and duplicate keys.	Confirmed

Sync Step	Design Requirement	Status
Acknowledgement	Backend returns central record id and accepted status for each synced item.	Confirmed
Retry	Retryable failures remain in queue with error message and retry count.	Confirmed
Conflict	Conflicts are detected and logged instead of silently overwriting records.	Confirmed
Conflict resolution	The exact rule for conflicting edits must be confirmed before final implementation.	Pending Decision

Duplicate Prevention

- Each offline record should have a local_uuid generated before first save.
- The backend should store the local_uuid, device identifier if used, user id, and shop id to detect duplicate submissions.
- Retries should update the same server record or return the previous acknowledgement instead of creating a second transaction.
- Sync logs should show accepted, rejected, pending, retrying, and conflict statuses.

User Interface Design

Platform	Main Screens	Design Notes	Status
Mobile app	Login, dashboard, product lookup, sales, purchases, expenses, debts, capital, stock, daily opening, daily closing, notifications, sync status.	Optimize for fast shop entry and offline confidence.	Confirmed
Web admin	Dashboard, shops, users, roles, products, purchases, sales, stock, expenses, capital, debts, reports, settings, audit trail.	Optimize for supervision, filtering, exports, and configuration.	Confirmed
Shared design	Search, filters, validation messages, empty states, loading states, permission states.	Keep wording consistent across mobile and web.	Proposed

Mobile Workflow Priorities

Open the business day.

Record sales quickly from product search or product list.

Record purchases, expenses, debts, repayments, stock adjustments, and capital entries where permission allows.

View current stock and low stock alerts.

See sync status clearly and retry failed records.

Close the business day with totals, notes, and exceptions.

Web Admin Workflow Priorities

Manage shops, users, roles, products, categories, and settings.

Review daily operations by shop and date.

Review sales, purchases, stock, expenses, debts, capital, profit, and allocation reports.

Inspect audit logs and synchronization problems.

Export reports if export is included in the release.

Security Error Handling and Reporting Design

Area	Design Requirement	Status
Authentication	Use secure password hashing and authenticated sessions or tokens.	Confirmed
Authorization	Check role, shop assignment, and permission at screen and API levels.	Confirmed
Transport	Use HTTPS in production.	Confirmed
Local data	Consider local database encryption or protected storage for sensitive mobile data.	Proposed
Audit	Log critical actions, changes, deletions, corrections, approvals, logins, and sync outcomes.	Confirmed
Validation	Validate required fields, numeric values, stock availability, valid dates, and duplicate codes.	Confirmed
Errors	Show clear user-facing errors and keep technical details in admin logs.	Confirmed
Reports	Reports should filter by date, shop, product, category, user, and status where relevant.	Confirmed
Exports	PDF or spreadsheet export for reports is recommended but can be prioritized after core reports.	Proposed

Deployment Testing and Open Design Decisions

Deployment Design

Item	Recommended Design	Status
Environments	Development, testing or staging, and production environments.	Proposed
Database backups	Automated MySQL backups with restore testing before production.	Proposed
Mobile releases	Controlled app release process with version tracking.	Proposed
Backend logging	Central server logs for errors, sync issues, and audit support.	Confirmed
Monitoring	Basic uptime, API error, database, and sync failure monitoring.	Proposed

Testing Design

- Functional tests for authentication, shops, users, products, purchases, sales, stock, expenses, capital, debts, reports, and daily closing.
- Offline tests for saving sales, purchases, expenses, debts, capital entries, stock adjustments, and closing records where allowed.
- Synchronization tests for success, retry, duplicate prevention, conflict detection, and rejected records.
- Security tests for role restrictions, shop restrictions, password handling, and unauthorized API access.
- Report tests to verify totals by shop, date, product, category, user, and payment status.
- Accounting tests should wait for owner-approved profit, allocation, expense, stock cost, tax, and debt rules.

Open Design Decisions

Decision	Reason	Status
Profit formula	Affects profit reports, dashboards, and 60/40 allocation source amount.	Pending Decision
Stock cost method	Affects cost of goods sold and profit on sales.	Pending Decision

Decision	Reason	Status
60/40 destination labels	Affects allocation reports and business interpretation.	Pending Decision
Allocation timing	Affects whether allocation is daily, weekly, monthly, per closing, or approval-based.	Pending Decision
Expense treatment	Affects net profit and business performance reports.	Pending Decision
Debt types	Affects customer debt, supplier debt, owner debt, and reporting design.	Pending Decision
Returns refunds discounts	Affects sales corrections, stock returns, and profit calculation.	Pending Decision
Negative stock policy	Affects sales validation and stock accuracy.	Pending Decision
Sync conflict policy	Affects offline edits and server changes.	Pending Decision
Tax or VAT handling	Affects invoices, reports, and profit calculations.	Pending Decision

SRS to SDD Traceability

SRS Area	SDD Design Coverage	Status
Mobile app and web admin	Presentation layer, UI design, platform responsibilities, shared backend API.	Confirmed
Super Admin and Shop Admin	Role permissions, shop assignment, screen access, API authorization.	Confirmed
Products purchases sales stock	Catalog, transaction, stock, stock movement, and reporting modules.	Confirmed
Expenses capital debts	Financial modules and data entities with pending accounting rule handling.	Confirmed
Profit and 60/40	Configurable calculation design with unresolved rules marked Pending Decision.	Pending Decision
Daily opening and closing	Daily session entity, mobile workflow, closing report, audit correction design.	Confirmed
Offline SQLite and MySQL sync	Local queue, sync service, duplicate prevention, acknowledgements, conflict detection.	Confirmed
Notifications reports audit trail	Notification layer, report service, audit service, filtering, export recommendation.	Confirmed
Security validation errors	Authentication, authorization, validation, error response, logging, local data considerations.	Confirmed